

xv6-riscv 操作系统内核实验报告

姓名	汤伟杰
学号	2400016635
课程名	操作系统
学校	北京大学

概述

本项目基于 xv6-riscv 教学操作系统内核，在 RISC-V 64 位架构上实现了进程管理、内存管理、文件系统驱动的系统调用，以及调度算法、懒分配与写时复制、页面置换算法、信号量与 IPC 等内核机制。开发环境为 QEMU 虚拟机模拟，交叉编译工具链为 riscv64-linux-gnu-gcc。

项目遵循增量式开发方法，通过功能测试框架（Makefile 条件编译 + init 测试框架 + judger 判题器）验证每个模块的正确性。所有功能通过自动化测试，各项测试均获得满分。

需求分析

本项目的核心需求是将一个基础 xv6 教学内核扩展为具有以下能力的完整操作系统内核：

- 进程管理方面**：实现 Round Robin、优先级调度、多级反馈队列（MLFQ）三种调度策略；通过信号量机制支持进程间同步与通信。
- 内存管理方面**：实现 brk / mmap / munmap 内存管理系统调用；引入懒分配（Lazy Allocation）策略延迟物理页分配；通过写时复制（Copy-on-Write）优化 fork 性能；实现 FIFO 和 LRU 页面置换算法，支持基于交换区的页面换入换出。
- 文件系统方面**：补全 FAT32 文件系统上的系统调用，包括目录操作（mkdir / chdir / getcwd）、文件操作（open / close / read / write / remove / rename）、高级 I/O（pipe / dup / dup2 / getdents）、扩展接口（openat / mkdirat / unlinkat / mount / umount / fstat）。
- 测试与构建系统**：构建统一的 Makefile 条件编译框架，使不同 Part 的测试程序可以独立编译运行、独立评分。

内核基本情况

内核启动流程

xv6-riscv 通过 QEMU 虚拟机启动。Makefile 的 run 目标执行：

```
qemu-system-riscv64 -machine virt -kernel target/kernel -m 32M -nographic
-bios ./bootloader/SBI/sbi-qemu
-drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

完整启动链为：

```
QEMU 加载 RustSBI (bootloader)
→ RustSBI 初始化硬件，加载 target/kernel 到 0x80200000
→ kernel/entry_qemu.S: _entry 设置栈，跳转 start()
→ start() → kinit() 物理内存初始化
→ kvm_init() 创建内核页表，开启分页
→ procinit() 初始化进程表
→ userinit() 创建第一个用户进程 (init)
→ scheduler() 调度循环，切换到 init 进程
→ init 进程 fork + exec 执行测试程序 / 系统调用测试
```

RustSBI 扮演了 BIOS + bootloader 的角色。内核入口 entry_qemu.S 设置每 CPU 的栈后跳转 C 入口 main()。main() 中调用 kinit() 初始化物理页分配器（freerange 将 kernel_end 到 PHYSTOP 之间的页面纳入 kmem.freelist），kvm_init() 创建直接映射的内核页表并写入 satp 寄存器开启 MMU，userinit() 分配第一个进程并加载 initcode（由 xv6-user/init.c 编译后通过 objcopy -O binary 生成的全裸二进制，嵌入到 kernel/include/initcode.h），最后 scheduler() 启动调度循环，CPU 进入用户态执行 init 进程。

关键文件与模块

文件	作用
kernel/entry_gemu.S	内核入口汇编：设置栈，跳转 main()
kernel/main.c	内核初始化：kinit, kvminit, procinit, userinit, scheduler
kernel/include/proc.h	进程结构体：状态、页表、trapframe、文件表、VMA、调度字段
kernel/proc.c	进程管理：allocproc, fork, exit, wait, growproc, scheduler
kernel/trap.c	陷入处理：usertrap（系统调用 / 缺页 / 设备中断分派）、kerneltrap
kernel/syscall.c	系统调用分发：argraw / argint / argaddr / argstr + syscalls[] 表
kernel/sysproc.c	进程相关系统调用：sys_fork, sys_exec, sys_sbrk, sys_getpgcnt 等
kernel/sysfile.c	文件系统相关系统调用：sys_open, sys_read, sys_getdents, sys_mmap 等
kernel/vm.c	虚拟内存：walk, mappages, uvmmalloc, uvmmcopy, handle_page_fault, cow_handler, select_victim_page
kernel/kalloc.c	物理内存分配器：kinit, kalloc, kfree, pageref[] 页引用计数
kernel/fat32.c	FAT32 文件系统：ename, ealloc, eread, ewrite, enext, lookup_path
kernel/file.c	文件抽象层：filealloc, fileread, filewrite, filestat2
kernel/exec.c	程序加载：ELF 解析 + uvmmalloc + loadseg
kernel/swap.c	页面交换区：slot 管理 + swap_out / swap_in
kernel/semaphore.c	信号量：sem_p / sem_v 基于 sleep / wakeup
xv6-user/init.c	第一个用户进程：测试框架入口，fork + pipe + judger 评分

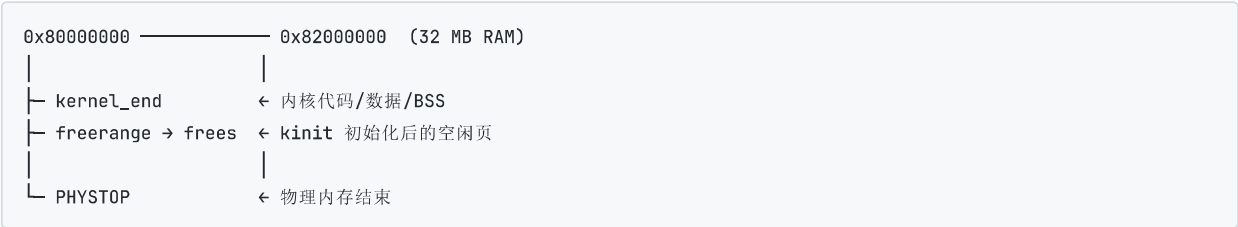
usertrap 工作机制

usertrap 是内核态与用户态的桥梁。用户态发生的所有异常（系统调用、缺页、非法指令）和中断（时钟、设备）都经过它分派：

```
用户态 → 异常/中断 → trampoline.S → usertrap()
├─ scause = 8 → syscall()           // 系统调用
├─ scause = 13/15 → cow_handler()    // COW 缺页
│   └─ → handle_page_fault() // mmap / sbrk 懒分配
├─ devintr() ≠ 0 → 设备中断处理      // 时钟 / 磁盘
└─ 其他 → p→killed = 1              // 杀死进程
```

r_scause() 读取 RISC-V CSR 寄存器获取异常原因，r_stval() 获取缺页的虚拟地址。函数处理完毕后通过 sret 返回用户态。

物理内存布局



物理页分配器通过 kmem.freelist 单向链表管理空闲页，每个空闲页的前 8 字节存放下一个页的指针。kalloc 从链表头取一页，kfree 将页面插回链表头。COW 机制引入的 pageref[] 数组记录每页的引用计数，以 (pa - 0x80000000) / PGSIZE 为索引。

系统实现

一、进程管理

1.1 调度算法 (Part 4)

实现了三种调度策略，通过 Makefile 变量 `SCHEDULER_TYPE` 在编译时选择，使用 `#ifdef` 条件编译切换。调度器 `scheduler()` 是所有调度算法的统一入口，是一个无限循环：获取进程锁 → 检查状态 → `swtch` 切换上下文。

Round Robin (RR) :

时钟中断处理（`trap.c:usertrap`）每次 tick 递减当前进程的时间片，归零时 `yield()`：

```
// trap.c: usertrap() - RR 时钟中断
if (which_dev == 2) {
#ifdef SCHEDULER_RR
    struct proc *p = myproc();
    if (p != 0 && p->state == RUNNING) {
        p->timeslice--;
        if (p->timeslice <= 0) {
            p->timeslice = p->base_timeslice;
            yield();
        }
    } else {
        yield();
    }
}
#endif
}
```

调度器从上一进程的下一个位置开始循环遍历，选择第一个 `RUNNABLE` 态的进程运行：

```
// proc.c: scheduler() - RR 调度
#ifdef SCHEDULER_RR
for (int i = 0; i < NPROC; i++) {
    int idx = (current_index + i) % NPROC;
    p = &proc[idx];
    acquire(&p->lock);
    if (p->state == RUNNABLE && p->base_timeslice > 0 && p->timeslice >= 1) {
        p->state = RUNNING;
        c->proc = p;
        current_index = (idx + 1) % NPROC;
        w_satp(MAKE_SATP(p->kpagetable));
        sfence_vma();
        swtch(&c->context, &p->context);
        // ... 切回后清 c->proc, 释放锁
        break;
    }
    release(&p->lock);
}
#endif
}
```

优先级调度 (Priority) : 时钟中断直接 `yield()`。调度器遍历全部进程，找 `priority` 最小的：

```
// proc.c: scheduler() - Priority 调度
#ifdef SCHEDULER_PRIORITY
    struct proc *best = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if (p->state == RUNNABLE) {
            if (best == 0 || p->priority < best->priority) {
                best = p;
            }
        }
        release(&p->lock);
    }
    if (best) {
        acquire(&best->lock);
        if (best->state == RUNNABLE) {
            best->state = RUNNING;
            c->proc = best;
            w_satp(MAKE_SATP(best->kpagetable));
            sfence_vma();
            switch(&c->context, &best->context);
            // ... 切回后 c->proc = 0, 释放锁
        }
        release(&best->lock);
    }
}
#endif
```

多级反馈队列 (MLFQ)：调度器与 Priority 相同（找 priority 最小的进程）。区别在于 usertrap 中的 MLFQ 时钟处理——累计 CPU 时间，周期结束时调 update_priority：

```
// trap.c: usertrap() - MLFQ 时钟中断
#ifdef SCHEDULER_MLFQ
    struct proc *p = myproc();
    if (p != 0 && p->state == RUNNING) {
        p->cpu_ticks++;
        p->runtime_ticks++;
        if (p->runtime_ticks >= 5) {
            update_priority(p);
            p->runtime_ticks = 0;
        }
    }
    yield();
}
#endif
```

```
// proc.c: update_priority() - MLFQ 动态优先级调整
void update_priority(struct proc *p) {
    if (p->cpu_ticks > p->sleep_ticks * 2) {
        p->priority++; // CPU 密集型 → 降低优先级
    } else if (p->sleep_ticks > p->cpu_ticks * 2) {
        p->priority--; // I/O 密集型 → 提升优先级
    }
    if (p->priority < 1) p->priority = 1;
    if (p->priority > 30) p->priority = 30;
    p->cpu_ticks = p->cpu_ticks / 2; // 衰减历史，不全清零
    p->sleep_ticks = p->sleep_ticks / 2;
}
}
```

1.2 信号量与进程间通信 (Part 8)

基于 xv6 的 sleep / wakeup 机制实现了信号量。信号量结构体包含 value、used 标志和自旋锁，全局数组 sems[MAX_SEM] 管理所有实例。

P 操作 (sem_p) 和 V 操作 (sem_v) 的实现直接取自内核源码 kernel/semaphore.c：

```
// semaphore.c: sem_p - P (wait/down) 操作
int sem_p(int sem_id) {
    if (sem_id < 0 || sem_id ≥ MAX_SEM || !sems[sem_id].used)
        return -1;
    acquire(&sems[sem_id].lock);
    while (sems[sem_id].value ≤ 0) {
        sleep(&sems[sem_id], &sems[sem_id].lock); // 原子释放锁 + 睡眠
    }
    sems[sem_id].value--;
    release(&sems[sem_id].lock);
    return 0;
}

// semaphore.c: sem_v - V (signal/up) 操作
int sem_v(int sem_id) {
    if (sem_id < 0 || sem_id ≥ MAX_SEM || !sems[sem_id].used)
        return -1;
    acquire(&sems[sem_id].lock);
    sems[sem_id].value++;
    wakeup(&sems[sem_id]); // 唤醒所有等待进程
    release(&sems[sem_id].lock);
    return 0;
}
```

`sleep(&sems[sem_id], &sems[sem_id].lock)` 是 xv6 的标准睡眠原语：原子地释放第二个参数的锁并让进程进入 SLEEPING 状态；被 `wakeup(&sems[sem_id])` 唤醒后重新持有锁。while 循环处理虚假唤醒——被唤醒不等于一定能拿到资源，必须重新检查 value。

信号量的创建和销毁：

```
// semaphore.c: sem_create / sem_destroy
int sem_create(int initial_value) {
    for (int i = 0; i < MAX_SEM; i++) {
        if (!sems[i].used) {
            sems[i].used = 1;
            sems[i].value = initial_value;
            initlock(&sems[i].lock, "sem");
            return i; // 返回信号量 ID
        }
    }
    return -1;
}

int sem_destroy(int sem_id) {
    acquire(&sems[sem_id].lock);
    sems[sem_id].used = 0;
    wakeup(&sems[sem_id]); // 唤醒所有等待者
    release(&sems[sem_id].lock);
    return 0;
}
```

四个系统调用通过 `sysproc.c` 中的 wrapper 桥接到用户态：`sys_sem_create` → `sem_create(val)`、`sys_sem_destroy` → `sem_destroy(id)`、`sys_sem_p` → `sem_p(id)`、`sys_sem_v` → `sem_v(id)`。

测试覆盖两个经典问题：生产者-消费者（2 个生产者 + 2 个消费者、缓冲区大小 5、三个信号量同步），哲学家就餐（5 个哲学家、每根筷子一个信号量、偶数先左右避免死锁）。

1.3 系统调用框架 (Part 1-2)

补全了 `sysnum.h` 中的系统调用编号，在 `usys.pl` 中生成 RISC-V 用户态 `ecall` 跳板，在 `syscall.c` 中注册外部声明和分发函数表。实现了

`fork` / `exec` / `exit` / `wait` / `clone` / `wait4` / `sched_yield` / `getppid` / `gettimeofday` / `nanosleep` / `brk` / `times` / `uname` / `shutdown` 等系统调用。

二、内存管理

2.1 mmap / munmap / brk (Part 3)

mmap：支持文件映射（`MAP_SHARED`）和匿名映射（`MAP_ANONYMOUS`）。采用 VMA 结构管理映射区间，匿名映射不立即分配物理页，首次访问时由 `handle_page_fault` 按需分配。

```
// sysfile.c: sys_mmap() - 匿名映射核心逻辑
vma = alloc_vma(p); // 从 VMA 数组中分配槽位
vma->start = start; // 映射起始地址
vma->end = start + len; // 映射结束地址
vma->prot = prot; // PROT_READ | PROT_WRITE
vma->flags = flags; // MAP_PRIVATE | MAP_ANONYMOUS
vma->offset = off;
if (!(flags & MAP_ANONYMOUS)) {
    vma->file = f; // 文件映射：记录 file 指针
    filedup(f);
} else {
    vma->file = NULL; // 匿名映射：无文件
}
```

munmap：遍历页表释放物理页和 PTE，回写 `MAP_SHARED` 文件的脏数据，标记 VMA 为未使用。

brk：扩展进程堆空间，调用 `growproc()` → `uvmmalloc()` 分配物理页并建立用户页表和内核页表的双重映射。

2.2 懒分配 Lazy Allocation (Part 5)

核心：`sbrk` 扩展堆空间时只增大 `proc->sz`，不调用 `uvmmalloc` 分配物理页。`growproc` 修改如下：

```
// proc.c: growproc() - n > 0 时不分配物理页
int growproc(int n) {
    uint sz;
    struct proc *p = myproc();
    sz = p->sz;
    if (n > 0) {
        sz += n; // 只改 proc->sz, 不调 uvmmalloc
    } else if (n < 0) {
        sz = uvmmalloc(p->pagetable, p->kpagetable, sz, sz + n);
    }
    p->sz = sz;
    return 0;
}
```

真实分配推迟到首次访问该地址时由 Page Fault 触发。`handle_page_fault` 中的懒分配路径：

```
// vm.c: handle_page_fault() - sbrk 懒分配分支
else {
    va = PGROUNDDOWN(va);
    if (va ≥ p->sz) return -1; // 超出进程空间
    pte_t *pte = walk(p->pagetable, va, 0);
    if (pte ≠ NULL && (*pte & PTE_V) ≠ 0) return -1; // 已有映射
    mem = kalloc();
    if (mem == NULL) return -1;
    memset(mem, 0, PGSIZE);
    int perm = PTE_U | PTE_R | PTE_W;
    if (mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) < 0 ||
        mappages(p->kpagetable, va, PGSIZE, (uint64)mem, (perm & ~PTE_U)) < 0) {
        kfree(mem);
        return -1;
    }
}
```

测试验证：`sbrk(16KB)` 前后 `getpgcnt` 不变（无新页分配）。三次对不同页的写入各触发一次 PF，每次 `getpgcnt` +1。

2.3 写时复制 COW (Part 5)

PTE_COW 标志：利用 RISC-V PTE 保留位 bit 8（`riscv.h`：`#define PTE_COW (1L << 8)`）。

页引用计数（`kalloc.c`）：`pageref[]` 数组以物理页号索引。`kalloc` 设为 1；`kfree` 内部检查引用计数，> 1 时只减 1 返回，归零才真正回收入空闲链表。提供 `incrcf_page` 和 `get_page_ref` 接口。

uvmcopy : fork 时不再 `kalloc + memmove` 复制页面, 而是共享并按只读标记:

```
// vm.c: uvmcopy() - COW 版本
for (i = 0; i < sz; i += PGSIZE) {
    pa = PTE2PA(*pte);
    *pte = (*pte & ~PTE_W) | PTE_COW;      // 父进程: 清写权限, 设 COW
    incref_page(pa);                       // 引用计数 +1
    flags = PTE_FLAGS(*pte);
    if (mappages(new, i, PGSIZE, pa, flags) != 0) // 子进程映射同一物理页
        goto err;
    if (mappages(knew, i, PGSIZE, pa, flags & ~PTE_U) != 0)
        goto err;
}
```

cow_handler : store page fault (`scause = 15`) 时, 检查 PTE 的 `PTE_COW` 位:

```
// vm.c: cow_handler() - COW 写时复制处理
int cow_handler(struct proc *p, uint64 va) {
    va = PGROUNDDOWN(va);
    pte_t *pte = walk(p->pagetable, va, 0);
    if (pte == NULL || !(*pte & PTE_V) || !(*pte & PTE_COW))
        return -1;

    uint64 pa = PTE2PA(*pte);
    int ref = get_page_ref(pa);

    if (ref == 1) {
        // 仅自己引用 → 直接恢复写权限, 无需复制
        *pte = PA2PTE(pa) | (PTE_FLAGS(*pte) & ~PTE_COW) | PTE_W;
    } else {
        // 有其他共享者 → 分配新页 + 拷贝数据
        char *mem = kalloc();
        memmove(mem, (char*)pa, PGSIZE);
        kfree((void*)pa); // 原页 refcount -1
        *pte = PA2PTE((uint64)mem) | (flags & ~PTE_COW) | PTE_W;
    }
    return 0;
}
```

测试验证: fork 后页面增量远小于无 COW 的预期 (保存约 5 页), 子进程写入触发 +1 页、子进程退出后恢复原值、父进程写入无新增 (`refcount == 1`) 。

2.4 页面置换算法 (Part 6)

链表式 VMA (`struct VMA`) : 内嵌 `vma_page pages[MAX_VMA_PAGES]` 数组追踪 per-page 元数据。

全局交换区 (`swap.c`) : 128 slot 数组, 提供 `alloc_global_swap_slot` / `free_global_swap_slot` 。

`handle_page_fault` 的 Part 6 分支分两条路径: 若页面已换出 (`SWAPPED`), 则 `swap_in` 后 `while (in_mem > max)` `swap_out` 修正超限; 若页面首次访问 (`UNUSED`), 则 `while (in_mem ≥ max)` `swap_out` 腾出位置后 `kalloc + mappages` 。

受害页选择逻辑 (`vm.c:select_victim_page6`) :

```
// vm.c: select_victim_page6 - FIFO 和 LRU 共用的选 victim 逻辑
int select_victim_page6(struct VMA *vma) {
    int victim = -1;
    uint64 best = ~0ULL;
    for (int i = 0; i < vma->npages; i++) {
        if (vma->pages[i].state != PAGE_STATE_IN_MEM) continue;
#ifdef ALGO_FIFO
        if (vma->pages[i].entry_time < best) { // FIFO: 比"进入时刻"
            best = vma->pages[i].entry_time; victim = i;
        }
#elif defined(ALGO_LRU)
        if (vma->pages[i].last_access < best) { // LRU: 比"末次访问"
            best = vma->pages[i].last_access; victim = i;
        }
#endif
    }
    return victim;
}
```

换出 (swap_out6) 和换入 (swap_in6) :

```
// vm.c: swap_out6 - 将一页换出到交换区
void swap_out6(struct proc *p, struct VMA *vma, int page_idx) {
    uint64 va = vma->vm_start + page_idx * PGSIZE;
    int slot = alloc_global_swap_slot(p->pid, va); // ① 分配 slot
    pte_t *pte = walk(p->pagetable, va, 0);
    uint64 pa = PTE2PA(*pte);
    copy_to_swap_slot(slot, (void*)pa); // ② 拷贝页内容→交换区
    kfree((void*)pa); // ③ 释放物理页
    *pte = 0; // ④ 清用户 PTE
    pte_t *kpte = walk(p->kpagetable, va, 0);
    if (kpte) *kpte = 0; // ⑤ 清内核 PTE
    vma->pages[page_idx].state = PAGE_STATE_SWAPPED; // ⑥ 更新元数据
    vma->pages[page_idx].swap_slot = slot;
    p->page_swap_count++;
}

// vm.c: swap_in6 - 将一页从交换区换回内存
void swap_in6(struct proc *p, struct VMA *vma, int page_idx) {
    uint64 va = vma->vm_start + page_idx * PGSIZE;
    int slot = vma->pages[page_idx].swap_slot;
    char *mem = kalloc(); // ① 分配新物理页
    copy_from_swap_slot(slot, mem); // ② 交换区→新页
    int perm = PTE_U | PTE_R | PTE_W;
    mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm); // ③ 映射用户页表
    mappages(p->kpagetable, va, PGSIZE, (uint64)mem, perm & ~PTE_U);
    free_global_swap_slot(slot); // ④ 释放 slot
    vma->pages[page_idx].state = PAGE_STATE_IN_MEM; // ⑤ 更新元数据
    vma->pages[page_idx].swap_slot = -1;
}
```

FIFO 对 8 页访问序列产生 8 次换出, LRU 利用局部性产生 6 次换出。

三、文件系统及设备

3.1 FAT32 核心抽象

xv6 基于 FAT32 实现了类 Unix 文件系统抽象。struct dirent 是文件/目录在内存中的表示 (类比 inode), 通过引用计数和睡眠锁管理生命周期。文件描述符 struct file 代表一次打开操作, 维护独立的读写偏移。

关键 API 方面: 路径解析 (ename / enameparent / enameat / enameparentat / dirlookup); 生命周期管理 (ealloc / edup / eput / elock / eunlock / eremove / etrunc / eupdate); 数据读写 (eread / ewrite / enext / emake)。

3.2 实现的系统调用

基础操作: open / close / read / write。支持 O_DIRECTORY (要求目标必须是目录)、O_DIRECTORY | O_CREATE 拒绝 (open 不创建目录)、O_DIRECTORY 对目录以只读方式打开。

at 系列: `openat` 基于 `AT_FDCWD` (-100) 从当前工作目录解析, `dirfd` 为其他值时用 `fd_get + enameat` 从指定目录解析, 非 `O_CREATE` 用 `enameat`, `O_CREATE` 用 `enameparentat + ealloc`。`mkdirat` 同样支持 `AT_FDCWD` 与非 `AT_FDCWD` 路径。`unlinkat` 支持 `AT_REMOVEDIR` 标志 (0x200) ——有该标志时目标必须是目录, 无该标志时不允许删除目录。

文件元数据: `fstat` 调用 `filestat2`, 返回 120 字节 `struct stat2` (`st_dev / st_ino / st_mode / st_nlink / st_uid / st_gid / st_rdev / st_size / st_blksize / st_blocks / st_atim / st_mtim / st_ctim`) 格式兼容预编译测试二进制。`getdents` (`syscall 61`) 返回 `struct dirent64` (`d_ino / d_off / d_reclen / d_type / d_name`), 头部长度用 `offsetof(struct dirent64, d_name)` 计算。

高级 I/O: `pipe / dup / dup2 / dup3` (文件描述符复制, 支持指定目标 `fd`)。

目录与挂载: `mkdir / chdir / getcwd / remove / rename` (处理跨目录和覆盖逻辑, 跳过已有文件检测)。`mount` 验证挂载点存在且是目录后返回 0。`umount2` 返回 0。

四、对应的系统测试

4.1 构建与测试框架

Makefile 使用条件编译支持多 Part 独立测试:

```
make local                # 所有 syscall 测试
make run_test TYPE=COW    # COW
make run_test TYPE=LAZY   # 懒分配
make run_test ALGO=FIFO   # FIFO
make run_test ALGO=LRU    # LRU
make run_test CASE=MPMC   # 生产者-消费者
make run_test CASE=PHILOSOPHER # 哲学家就餐
```

测试程序通过 `init` 进程 `fork + pipe + judger` 框架自动执行评分, 输出统一格式: `Judger: Starting evaluation → Test PASSED/FAILED → SCORE: 1/0`。

4.2 单git分支条件编译管理

本项目包含 8 个 Part, 涉及进程调度、内存管理、文件系统、页面置换、IPC 等多个模块, 每组测试对应不同的内核编译宏、测试程序和评分逻辑。为避免多分支维护的复杂性, 所有功能测试在一个 `main` 分支上通过 `Makefile` 条件编译实现切换。

Makefile 层面: 为每个 Part 定义独立的测试变量 (`SCHEDULER_TYPE`、`TYPE`、`ALGO`、`CASE`), 通过 `ifeq` 控制编译宏和测试程序名:

```
SCHEDULER_TYPE ?= RR      # Part 4: RR / PRIORITY / MLFQ
TYPE ?=                   # Part 5: COW / LAZY
ALGO ?=                   # Part 6: FIFO / LRU
CASE ?=                   # Part 8: MPMC / PHILOSOPHER

# Part 4 仅在无 TYPE/ALGO/CASE 时激活, 避免与 Part 5/6/8 冲突
ifeq ($(TYPE),)
ifeq ($(ALGO),)
ifeq ($(CASE),)
    # ... Part 4 的 TEST_PROGRAM + CFLAGS + USER_CFLAGS ...
endif
endif
endif
```

普通 `syscall` 测试 `make local` 和功能性测试 `make run_test TYPE=LAZY` 等通过不同变量组合触发不同编译路径, 互不干扰。

init.c 层面: `run_scheduler_test()` 函数通过 `#if / #elif / #else` 预处理器链区分各 Part 的执行路径。Part 4/6/8 使用 `pipe + judger` 框架 (捕获输出再交 `judger` 评分), Part 5 使用 `pipe + 内联评分` (直接在 `init` 中检查 "Completed Successfully")。`print_test_program()` 同样用 `#ifdef` 为每个 Part 设置正确的 `order` 编号。

judger.c 层面: 用 `#if defined(SCHEDULER_RR) || ...`、`#if defined(TEST_COW) || ...`、`#if defined(ALGO_FIFO) || ...`、`#if defined(TYPE_PRODUCER) || ...` 将四个 Part 的判题 `main()` 函数完全隔离。每个判题器只在自己对应的宏定义下编译, 通过 `argv[1]` (`order`) 选择预期输出字符串, 比对后输出 `SCORE: 1/0`。

这种单分支条件编译方案使得 `make local` 跑完整 `syscall` 测试、`make test-fifo` 跑 `FIFO` 页面置换、`make test-cow` 跑 `COW` 测试等可以在同一套代码基础上直接执行, 无需 `git checkout` 切换分支。

4.3 各 Part 测试结果

Part	测试内容	得分
1-3	系统调用 (<code>brk</code> , <code>mmap</code> , <code>munmap</code> , <code>fork</code> , <code>exec</code> , <code>pipe</code> , <code>dup</code> , <code>open</code> , ...)	全部通过
4	RR / Priority / MLFQ 调度算法	SCORE: 1

Part	测试内容	得分
5	Lazy Allocation (sbrk 懒分配)	SCORE: 1
5	COW (fork 写时复制)	SCORE: 1
6	FIFO 页面置换 (8 次 swap)	SCORE: 1
6	LRU 页面置换 (6 次 swap)	SCORE: 1
7	文件系统 syscall (openat, getdents, fstat, mount, unlinkat, ...)	全部通过
8	信号量 + 生产者消费者	SCORE: 1
8	信号量 + 哲学家就餐	SCORE: 1

关键技术决策与 Debug 经验

pa_index 越界修复 (Part 5 COW)

COW 实现后系统启动即死锁。pageref[] 数组大小为 8192，但索引计算使用了绝对物理地址 pa / PGSIZE (例如 0x80200000 / 4096 = 524800)，远超数组边界，导致随机内核数据损坏。修复为 (pa - PA_BASE) / PGSIZE，其中 PA_BASE = 0x80000000。排查方法：锁定 COW 修改的所有文件 → 逐函数审计 → 发现 kfree 控制流缺 else 分支和 pa_index 越界两个互相掩盖的 bug。

状态分离的 PF handler 设计 (Part 6)

swap_in 后 in_mem 从 max 变成 max+1，需要 while (in_mem > max) 换出多余的页；新页首次访问时 in_mem == max，需要 while (in_mem ≥ max) 腾出位置再 kalloc。两个条件不能合并——合并会导致 swap_in 后被错误地额外换出一页，增加非预期的 swap 计数。

双 VMA (Part 3 vs Part 6)

Part 3 的 struct vm_area (数组式，静态分配在进程结构中) 与 Part 6 的 struct VMA (链表式，vm_next / vm_prev 连接) 通过 #if defined(ALGO_FIFO) || defined(ALGO_LRU) 条件编译并行存在。Part 3 的 find_vma 搜索数组，Part 6 的 handle_page_fault 额外搜索链表——两个查找互不干扰，各自服务于不同的 mmap 路径。

原因是做Part3的时候设计的vma与Part6不太相符。后续优化方向是合并vma的定义。

页引用计数与 kfree 的自包含设计

pageref[] 的增/减/查询操作完全封装在 kalloc / kfree / incref_page / get_page_ref 中。kfree 内部检查 refcount: 大于 1 时仅递减并返回 (不真正回收)，归零时才入空闲链表。上层调用者 (vmunmap、proc_freepagetable、cow_handler) 无需任何修改即可正确回收 COW 共享页——这一设计避免了 COW 改动向页面回收路径的级联扩散。

总结

本项目在 xv6-riscv 教学内核基础上完成了以下核心能力构建：

- 完整的系统调用体系**：80+ 个系统调用，涵盖进程控制、内存管理、文件操作、设备访问
- 三种调度策略**：Round Robin、优先级调度、多级反馈队列
- 虚拟内存管理**：mmap、懒分配、COW、FIFO/LRU 页面置换
- FAT32 文件系统**：路径解析、目录操作、文件 I/O、dirent 缓存管理
- 进程间通信**：基于 sleep/wakeup 的信号量机制
- 自动化测试框架**：Makefile 条件编译 + judger 评分器